Software Architecture & Design Specification

Event Management System — SE Mini Project

Version 1.0

4 November 2026

Authors: Shravani Mallur (PES1UG24CS621) · Arjun Bhat (PES1UG24CS655) · Amogh Herle (PES1UG24CS653)

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design of the Event Management System (EMS). It describes the system's architectural pattern, its decomposition into components, how those components trace back to the Functional and Non-Functional Requirements in the Software Requirements Specification (SRS), the security architecture, and the detailed design of two representative workflows, the public API, and the error-handling conventions followed throughout the codebase. It is written for the development team and for evaluators assessing architectural and design-level decisions, as distinct from the requirements themselves.

1.2 Scope

EMS is a web application that lets Participants browse and register for campus events, lets Organizers create, edit and cancel events, and lets Admins manage user accounts, event categories and platform-wide statistics. This document covers the web application (Next.js App Router), its server-side API, its business-logic layer, and its PostgreSQL data store. It does not cover infrastructure provisioning or CI/CD pipeline configuration beyond what is needed to explain error-handling and deployment-relevant fallback behavior (see Section 4.5).

1.3 References

- Documents/SRS_DOCUMENT.pdf — Software Requirements Specification (10 FRs, 7 NFRs, traceability matrix)
- Documents/SECURITY_DOCUMENT.pdf — Security Requirements Specification (SRSec)
- Documents/TEST-PLAN_DOCUMENT.pdf — Test Plan Document (test strategy, traceability, security validation)
- Documents/COMPONENT_DIAGRAM.svg / USECASE_DIAGRAM.svg / USECASE_FLOW.pdf — supporting UML diagrams
- prisma/schema.prisma — authoritative data model

1.4 Definitions and Acronyms

Term	Meaning
FR / NFR	Functional Requirement / Non-Functional Requirement, as numbered in the SRS
RBAC	Role-Based Access Control (Admin, Organizer, Participant)
ORM	Object-Relational Mapper — Prisma, used for all database access
SERIALIZABLE tx	A PostgreSQL transaction run at the SERIALIZABLE isolation level
IDOR	Insecure Direct Object Reference
SMTP	Simple Mail Transfer Protocol — used to deliver all transactional email

2. Architectural Overview

2.1 Architectural Pattern

EMS follows a layered (N-tier) architecture implemented as a single deployable Next.js application (a "modular monolith" rather than a microservice decomposition, appropriate for the project's scale). Four layers are strictly ordered — each layer calls only the layer directly beneath it:

Layer	Responsibility	Representative modules
Presentation	React Server/Client Components rendering pages and forms; no business rules.	app/events/**, app/admin/**, app/components/**
API (Controller)	Next.js Route Handlers: parse/validate the request, call one business-logic function, map its result to an HTTP response.	app/api/**/route.ts
Business Logic	All domain rules — capacity checks, waitlist promotion, notification dispatch, RBAC checks, locking.	lib/registration.ts, lib/auth.ts, lib/categories.ts, lib/stats.ts, lib/export.ts
Data Access	Prisma ORM — typed queries and migrations against PostgreSQL.	prisma/schema.prisma, lib/prisma.ts

Two cross-cutting concerns sit alongside these layers rather than inside any one of them: an in-process per-event mutex (withEventLock) and SERIALIZABLE transactions, which together serialize every state-changing operation on a given event (Section 3.4); and a notification dispatcher (lib/emailTemplates.ts + lib/mailer.ts) invoked by the business-logic layer only after a change has committed, never before (Section 4.5).

2.2 Component Diagram

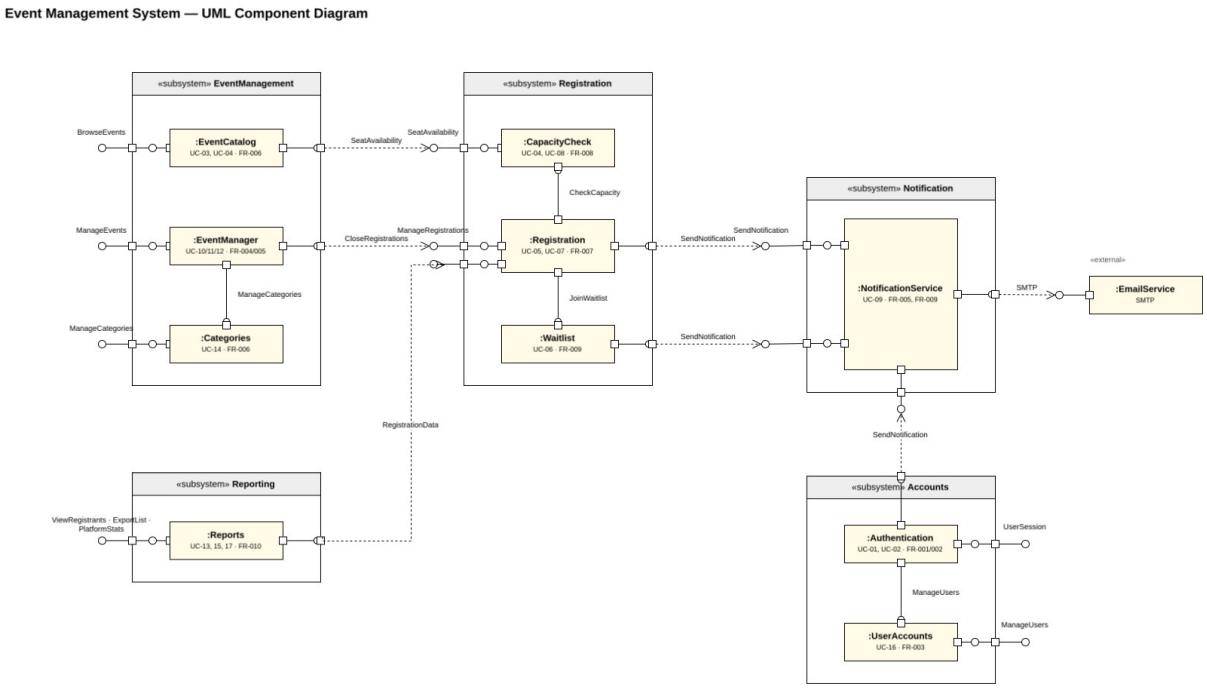


Figure 1 — UML component diagram: five subsystems (EventManagement, Registration, Notification, Reporting, Accounts), their provided/required interfaces, and the external SMTP dependency.

2.3 Component Descriptions

Subsystem	Component	Responsibility	Key files
EventManagement	EventCatalog	Browse, search and filter active events.	app/events/page.tsx, lib/eventFilters.ts
EventManagement	EventManager	Create, edit and cancel events (organizer-owned).	app/events/create, app/events/[id]/edit, app/api/events/[id]/route.ts
EventManagement	Categories	Admin-managed category list used by event forms and filters.	lib/categories.ts, app/admin/categories
Registration	CapacityCheck	Atomic seat-count enforcement at registration time.	lib/registration.ts (registerForEvent)
Registration	Registration	Register for / cancel a registration.	app/api/events/[id]/register, app/api/registrations
Registration	Waitlist	FIFO waitlist with auto-promotion on a freed seat.	lib/registration.ts (waitlist + promotion logic)

Subsystem	Component	Responsibility	Key files
Notification	NotificationService	Templated transactional email for every lifecycle event.	lib/emailTemplates.ts, lib/mailer.ts
Reporting	Reports	Participant CSV/PDF export; organizer and platform statistics.	lib/export.ts, lib/stats.ts
Accounts	Authentication	Signup, login, email verification, password reset, session.	lib/auth.ts, lib/session.ts, lib/tokens.ts
Accounts	UserAccounts	Admin view/suspend/delete of user accounts.	app/api/admin/users, app/admin/users

2.4 Traceability — Requirements to Architecture

Mirrors the SRS traceability matrix (Documents/SRS_DOCUMENT.pdf §3), restated against the components above.

FR	Description	Primary component(s)	Supporting NFR(s)
FR-001	Account creation + email verification	Authentication	NFR-003
FR-002	Login, password reset	Authentication	NFR-003, NFR-004
FR-003	Roles, profile update, admin user management	Authentication, UserAccounts	NFR-003, NFR-004
FR-004	Create / edit events	EventManager	NFR-005
FR-005	Cancel events, notify participants	EventManager, NotificationService	NFR-004, NFR-007
FR-006	Browse, search, filter, categorize events	EventCatalog, Categories	NFR-001, NFR-002, NFR-006
FR-007	Register for events	Registration	NFR-001, NFR-002, NFR-005, NFR-006
FR-008	Capacity check / overbooking prevention	CapacityCheck	NFR-001, NFR-002, NFR-006, NFR-007
FR-009	Waitlist, self-cancellation,	Waitlist,	NFR-002,

FR	Description	Primary component(s)	Supporting NFR(s)
	notifications	NotificationService	NFR-007
FR-010	Counts, export, statistics	Reports	NFR-001, NFR-002, NFR-005

3. Security Architecture

This section maps the security controls actually implemented in the codebase to Documents/SECURITY_DOCUMENT.pdf (SRSec).

3.1 Authentication and Session Management

- Passwords are hashed with bcrypt (bcryptjs), cost factor 12, never stored or logged in plain text — SRSec §2.1.
- A session is a cookie of the form userId.expiresAt.signature, where signature = HMAC-SHA256(SESSION_SECRET, userId.expiresAt). It is verified with a constant-time comparison (crypto.timingSafeEqual) to prevent timing attacks (lib/session.ts).
- The cookie is set HttpOnly; SameSite=Lax (and Secure outside local development), which stops script-based theft (XSS exfiltration of the cookie) and gives baseline CSRF protection for state-changing requests — SRSec §2.2.
- The session carries only a user id and an expiry — never a cached role or verified flag — so every request re-reads the live User row. A suspension or role change takes effect on the very next request rather than staying valid for the life of an old session.
- Verification and password-reset tokens are random, single-use (usedAt is set on consumption) and time-limited (expiresAt), stored as hashes (VerificationToken/PasswordResetToken), not as the raw token — SRSec §2.1.

3.2 Authorization (RBAC)

- Every state-changing route re-derives the caller's role from the database on the server (requireOrganizer / requireAdmin in lib/auth.ts); the client never asserts its own role for anything but a demo role-switcher header, which only governs the UI it shows.
- Ownership is checked explicitly wherever it matters: an organizer may only edit or cancel an event where event.organizerId equals their own id (app/api/events/[id]/route.ts), preventing one organizer from acting on another's event — an IDOR control, SRSec §3.3.
- Admin-only routes (app/api/admin/**) reject any non-admin caller with 403 before touching the database.

3.3 Data Protection

- All traffic is served over HTTPS/TLS 1.2+ at the hosting layer (Vercel); no route accepts credentials over plain HTTP — SRSec §4.1.

- Password hashes and token hashes are the only credential-shaped data persisted; PII (name, email) is stored as-is, consistent with the SRS's scope, with encryption at rest provided by the managed PostgreSQL instance — SRSec §4.2.
- All request bodies are validated against zod schemas (lib/validation.ts) before any field reaches the database — rejecting malformed input at the API boundary rather than relying on the ORM or the database to catch it.

3.4 Concurrency as an Integrity Control

Every mutation that can race another request on the same event — register, cancel a registration, edit an event, cancel an event — goes through two layers of defense in lib/registration.ts:

- withEventLock(eventId, fn) — an in-process async mutex, keyed by event id, that serializes concurrent calls for the same event within one server process.
- withSerializableTransaction(db, fn) — wraps the operation in a PostgreSQL SERIALIZABLE transaction (with a single retry on Prisma's P2034 serialization-failure code), which is what actually guarantees correctness across multiple server processes/instances, not just within one.

This combination is what the SRS's acceptance criteria for FR-007/FR-008/FR-009 and NFR-002 require: 100 simultaneous requests for a 1-seat event must confirm exactly one, and 100 duplicate requests from one user must create exactly one row. It also underwrites FR-005b: a cancellation's UPDATE ... WHERE status = 'ACTIVE' guard, run inside the same lock and transaction, is what guarantees participants are notified exactly once even if two cancel requests race.

3.5 Security Requirement Coverage

SRSec section	Requirement	Status	Where enforced
2.1 Password security	Salted bcrypt/Argon2 hashing	Met	app/api/auth/signup, reset-password (bcryptjs, cost 12)
2.2 Session management	HttpOnly/SameSite cookies	Met	lib/session.ts
2.3 Account lockout	Lock after repeated failed logins	Gap	Not implemented — tracked as part of NFR-004 (issue #59)
3.1/3.2 RBAC, least privilege	Server-side role + ownership checks	Met	lib/auth.ts, ownership checks per route
3.3 IDOR prevention	Server-side object-ownership checks	Met	app/api/events/[id]/route.ts and similar
4.1 Data in transit	TLS 1.2+	Met	Enforced at the hosting layer (Vercel)
4.2 Data at rest	Encryption of sensitive data	Met	Managed PostgreSQL encryption at rest
5.1 SQLi mitigation	Parameterized queries / ORM	Met	All access via Prisma — no raw SQL

SRSec section	Requirement	Status	Where enforced
5.2 XSS mitigation	Sanitize/escape user input	Met	escapeHtml() in lib/emailTemplates.ts; React escapes DOM output by default
5.3 CSRF mitigation	Anti-CSRF tokens	Partial	SameSite=Lax cookie only — no explicit per-request CSRF token
6.1/6.2 Audit logging	Security-event logging, sanitized	Gap	console.error on failures only — no durable, queryable audit log yet (tracked under NFR-004)

The gaps and partial items above are carried forward honestly rather than hidden — see Section 5 (Deviations) and backlog issues NFR-004 (#59) and NFR-007 (#62).

4. Detailed Design

4.1 Data Model

The persistent data model (prisma/schema.prisma) has six entities:

Entity	Key fields	Relationships
User	id, name, email (unique), passwordHash, role, verified, suspended	1—N Event (as organizer), 1—N Registration
Event	id, name, date, time, venue, category, categoryId, capacity, status	N—1 User (organizer), 1—N Registration, N—1 Category
Registration	id, userId, eventId, status (CONFIRMED | WAITLISTED | CANCELLED), reminderSentAt	N—1 User, N—1 Event; unique on (userId, eventId)
Category	id, name (unique)	1—N Event
VerificationToken	id, userId, tokenHash (unique), expiresAt, usedAt	N—1 User
PasswordResetToken	id, userId, tokenHash (unique), expiresAt, usedAt	N—1 User

The unique constraint on (userId, eventId) in Registration is what FR-007's duplicate-registration prevention is actually built on — a second registration attempt fails at the database level even if every application-level check were somehow bypassed.

4.2 Sequence Diagram 1 — Register for an Event

Covers FR-007 (registration), FR-008 (capacity check) and FR-009a (waitlist). Traced from app/api/events/[id]/register/route.ts and lib/registration.ts.

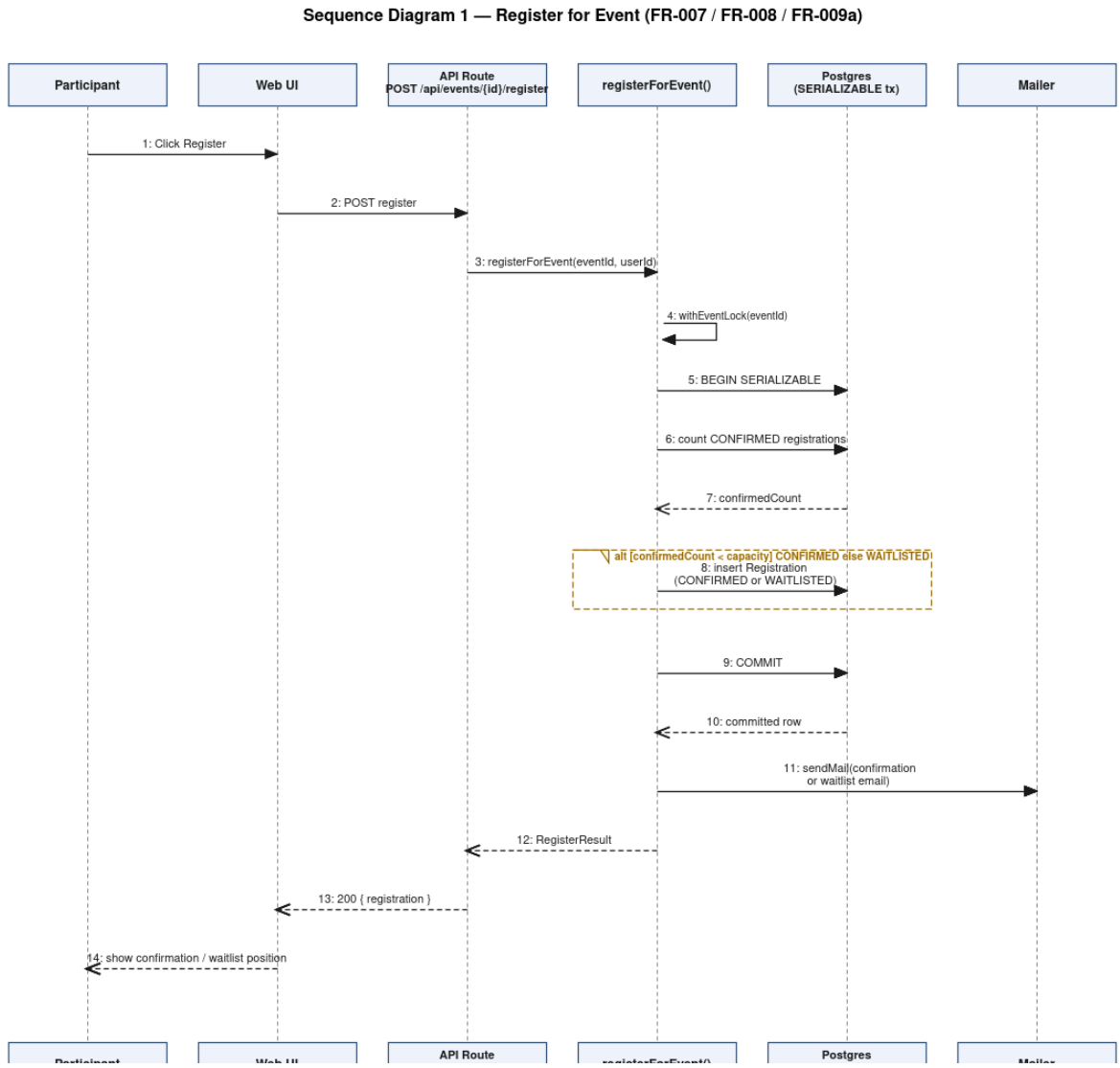


Figure 2 — Registering for an event: the lock/transaction pair decides CONFIRMED vs. WAITLISTED atomically, and the confirmation/waitlist email is sent only after commit.

4.3 Sequence Diagram 2 — Organizer Cancels an Event

Covers FR-005a (cancellation) and FR-005b (participant notification). Traced from app/api/events/[id]/route.ts (PATCH) and lib/registration.ts (cancelEvent, notifyEventParticipants).

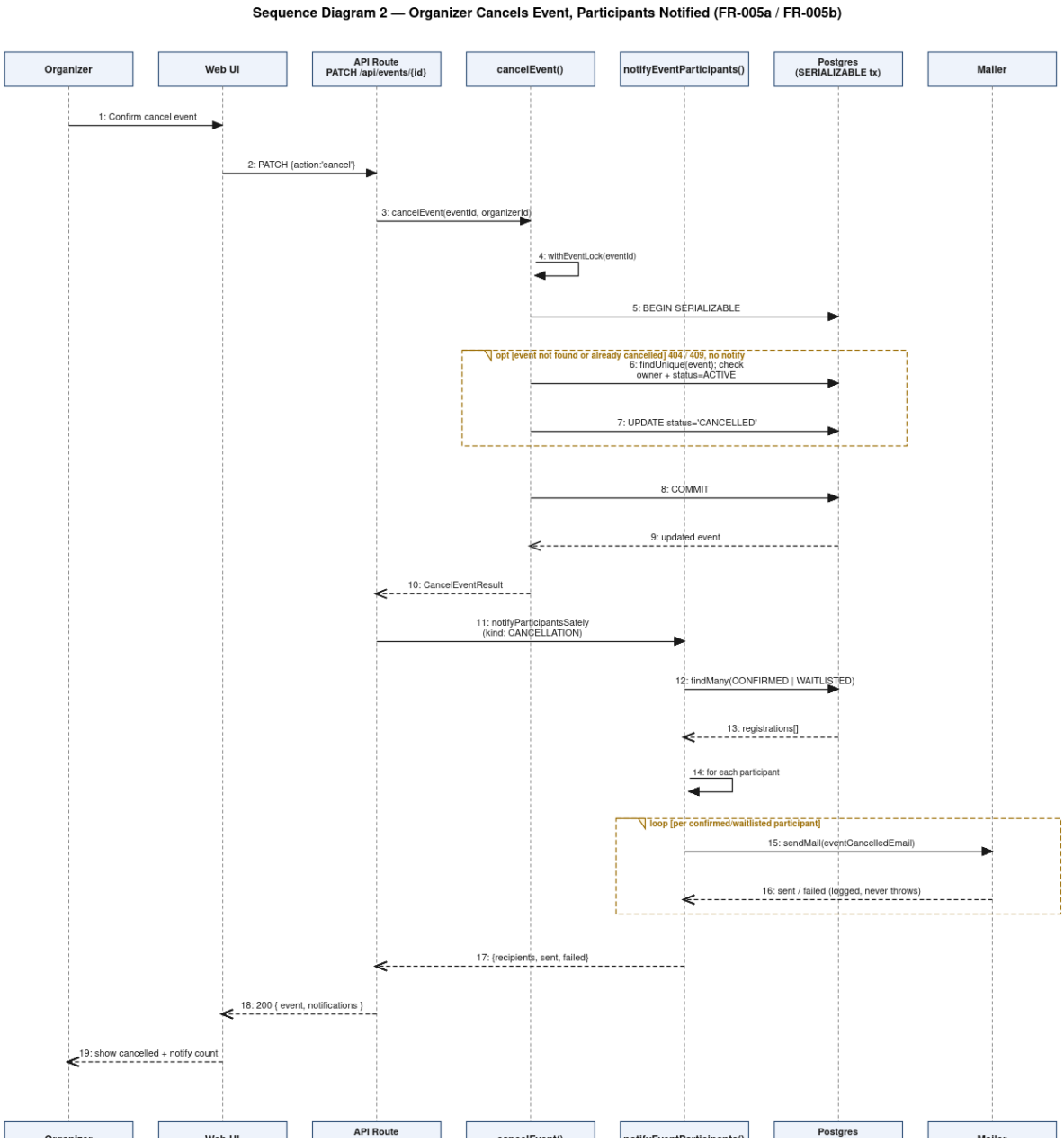


Figure 3 — Cancelling an event: the status=ACTIVE guard inside the locked transaction stops two concurrent cancels from both succeeding, so the notification loop that follows runs at most once.

4.4 API Design

All endpoints are under /api, implemented as Next.js Route Handlers. Request/response bodies are JSON. Auth column: "Organizer"/"Admin" means requireOrganizer/requireAdmin is enforced server-side; "Session" means a valid login session is required; "Public" means no authentication is required.

Endpoint	Methods	Auth	Purpose
/api/auth/signup	POST	Public	Create account, send verification email
/api/auth/verify	POST	Public	Consume an email-

Endpoint	Methods	Auth	Purpose
			verification token
/api/auth/login	POST	Public	Verify credentials, issue session cookie
/api/auth/logout	POST	Session	Clear session cookie
/api/auth/forgot-password	POST	Public	Issue a password-reset token by email
/api/auth/reset-password	POST	Public	Consume reset token, set new password
/api/auth/me	GET, PATCH	Session	Read / update the caller's own profile
/api/events	GET, POST	GET: Public · POST: Organizer	List/search active events · create an event
/api/events/{id}	GET, PATCH, PUT, DELETE	GET: Public · others: Organizer (owner)	Fetch · cancel (PATCH) · edit (PUT) · hard-delete an event
/api/events/{id}/register	GET, POST, DELETE	Session	Check / create / cancel a registration
/api/events/{id}/waitlist	GET, POST, DELETE	Session	Waitlist position / join / leave
/api/events/{id}/export	GET	Organizer (owner)	CSV/PDF export of an event's registrants
/api/registrations	GET, POST, DELETE	Session	A participant's own registrations across events
/api/categories	GET	Public	List active categories, for forms and filters
/api/reminders	GET, POST	Internal/cron	Dispatch upcoming-event reminder emails
/api/admin/users	GET	Admin	List all user accounts
/api/admin/users/{id}	PATCH, DELETE	Admin	Suspend/unsuspend, change role, or delete a user
/api/admin/categories	GET, POST	Admin	List / create categories
/api/admin/categories/{id}	PATCH, DELETE	Admin	Rename / delete a category

Endpoint	Methods	Auth	Purpose
/api/admin/statistics	GET	Admin	Platform-wide counts (events, users, registrations)

4.5 Error Handling Protocols

4.5.1 Response shape and status codes

Every route returns NextResponse.json(body, { status }). Errors are a flat { error: string } (plus field-level errors for validation failures), never an exception leaking a stack trace to the client. Status codes are used consistently across the codebase:

Status	Meaning in this codebase
400	Malformed JSON body or a field that fails zod validation
401	No valid session where one is required
403	Valid session, but wrong role or not the resource's owner
404	Referenced event/user/registration does not exist
409	A state conflict — already cancelled, capacity below confirmed count, stale optimistic-concurrency token (updatedAt mismatch)
500	An unexpected server/database error, logged via console.error and never exposed to the client in detail

4.5.2 Prisma error translation

Business-logic and route code translates a small, explicit set of Prisma error codes into the status codes above, rather than letting them surface as 500s:

Prisma code	Meaning	Handling
P2025	"Record to update/delete not found" — used deliberately as the result of a conditional WHERE (e.g. status: 'ACTIVE') that excludes the current row.	Mapped to 404 or 409 depending on the call site (e.g. a conditional cancel update failing this way means "already cancelled" → 409).
P2034	Transaction failed due to a write conflict under SERIALIZABLE isolation.	Retried exactly once by withSerializableTransaction; a second failure propagates as a 500.
P2021	Referenced table does not exist (used only for the optional Category relation, which some environments lack).	Caught narrowly and treated as "no category relation available" — the event is still saved with its plain category string.

Prisma code	Meaning	Handling
PrismaClientInitializationError	The database is unreachable at request time.	Several read/mutate paths fall back to an in-memory store (app/api/events/store.ts) so the demo and tests keep working without a live database; this fallback is explicitly out of scope for capacity-aware guarantees, which require the real database.

4.5.3 Notification failures never fail the primary action

A theme repeated across FR-005b, FR-009a/b and FR-010a: once the primary state change (cancel, register, promote, export) has committed, a failure to notify or email a participant is caught, logged, and reported back as a count (e.g. { recipients, sent, failed }) — it is never allowed to turn a successful mutation into an error response. notifyParticipantsSafely() in app/api/events/[id]/route.ts is the canonical example: it wraps notifyEventParticipants() in a try/catch and returns a zeroed result on failure rather than propagating the exception.

4.5.4 Optimistic concurrency on edits

FR-004b's edit endpoint accepts an optional expectedUpdatedAt from the client. If it is present and does not match the row's current updatedAt, the edit is rejected with 409 (EVENT_EDIT_CONFLICT_ERROR) instead of silently overwriting a newer change — applied identically on both the database path and the in-memory fallback.

5. Deviations from the Initial Design

Tracked here per the project's Sprint 2 process requirement to formally document any departure from the SRS/SAD for the final demo.

- FR-004b (event editing), FR-006b (search/filter) and FR-006c (category management) were merged into main, then reverted on 2026-10-29/30 because they had been merged without review — not because of a design defect. All three were re-merged via PR #47 on 2026-11-04 once resolved alongside FR-005b; the architecture described in this document reflects that final, re-reviewed state.
- NFR-007's 80% coverage target is tracked but not yet met — CI currently reports ~78% for the core modules (.github/workflows/ci.yml).
- SRSec §2.3 (account lockout after repeated failed logins) and §6.1/6.2 (durable, sanitized security audit log) are not yet implemented; both are tracked as part of NFR-004 (issue #59) rather than silently dropped.
- SRSec §5.3 asks for explicit anti-CSRF tokens; the current implementation relies on SameSite=Lax cookies only, which covers the common case but is narrower than a token-based scheme.